

PHP - Functions

PHP functions are similar to other programming languages. A function is a piece of code which takes one more input in the form of parameter and does some processing and returns a value.

You already have seen many functions like **fopen()** and **fread()** etc. They are built-in functions but PHP gives you option to create your own functions as well.

There are two parts which should be clear to you –

- ☐ Creating a PHP Function
- ☐ Calling a PHP Function

In fact you hardly need to create your own PHP function because there are already more than 1000 of built-in library functions created for different area and you just need to call them according to your requirement.

Please refer to [PHP Function Reference](#)  for a complete set of useful functions.

Creating PHP Function

Its very easy to create your own PHP function. Suppose you want to create a PHP function which will simply write a simple message on your browser when you will call it. Following example creates a function called `writeMessage()` and then calls it just after creating it.

Note that while creating a function its name should start with keyword **function** and all the PHP code should be put inside { and } braces as shown in the following example below –

[Live Demo](#)

```
<html>

<head>
  <title>Writing PHP Function</title>
</head>

<body>

  <?php
    /* Defining a PHP Function */
    function writeMessage() {
      echo "You are really a nice person, Have a nice time!";
    }

    /* Calling a PHP Function */
    writeMessage();
  ?>

</body>
</html>
```

This will display following result –

```
You are really a nice person, Have a nice time!
```

PHP Functions with Parameters

PHP gives you option to pass your parameters inside a function. You can pass as many as parameters your like. These parameters work like variables inside your function. Following example takes two integer parameters and add them together and then print them.

[Live Demo](#)

```
<html>

<head>
  <title>Writing PHP Function with Parameters</title>
</head>

<body>

  <?php
    function addFunction($num1, $num2) {
      $sum = $num1 + $num2;
      echo "Sum of the two numbers is : $sum";
    }

    addFunction(10, 20);
  ?>

</body>
</html>
```

This will display following result –

```
Sum of the two numbers is : 30
```

Passing Arguments by Reference

It is possible to pass arguments to functions by reference. This means that a reference to the variable is manipulated by the function rather than a copy of the variable's value.

Any changes made to an argument in these cases will change the value of the original variable. You can pass an argument by reference by adding an ampersand to the variable name in either the function call or the function definition.

Following example depicts both the cases.

[Live Demo](#)

```
<html>
```

```
<head>
  <title>Passing Argument by Reference</title>
</head>

<body>

  <?php
    function addFive($num) {
      $num += 5;
    }

    function addSix(&$num) {
      $num += 6;
    }

    $orignum = 10;
    addFive( $orignum );

    echo "Original Value is $orignum<br />";

    addSix( $orignum );
    echo "Original Value is $orignum<br />";
  ?>

</body>
</html>
```

This will display following result –

```
Original Value is 10
Original Value is 16
```

PHP Functions returning value

A function can return a value using the **return** statement in conjunction with a value or object. return stops the execution of the function and sends the value back to the calling code.

You can return more than one value from a function using **return array(1,2,3,4)**.

Following example takes two integer parameters and add them together and then returns their sum to the calling program. Note that **return** keyword is used to return a value from a function.

[Live Demo](#)

```
<html>

  <head>
    <title>Writing PHP Function which returns value</title>
  </head>

  <body>
```

```
<?php
    function addFunction($num1, $num2) {
        $sum = $num1 + $num2;
        return $sum;
    }
    $return_value = addFunction(10, 20);

    echo "Returned value from the function : $return_value";
?>

</body>
</html>
```

This will display following result –

```
Returned value from the function : 30
```

Setting Default Values for Function Parameters

You can set a parameter to have a default value if the function's caller doesn't pass it.

Following function prints NULL in case use does not pass any value to this function.

[Live Demo](#)

```
<html>

<head>
    <title>Writing PHP Function which returns value</title>
</head>

<body>

    <?php
        function printMe($param = NULL) {
            print $param;
        }

        printMe("This is test");
        printMe();
    ?>

</body>
</html>
```

This will produce following result –

```
This is test
```

Dynamic Function Calls

It is possible to assign function names as strings to variables and then treat these variables exactly as you would the function name itself. Following example depicts this behaviour.

[Live Demo](#)

```
<html>

<head>
  <title>Dynamic Function Calls</title>
</head>

<body>

  <?php
    function sayHello() {
      echo "Hello<br />";
    }

    $function_holder = "sayHello";
    $function_holder();

  ?>

</body>
</html>
```

This will display following result –

Hello